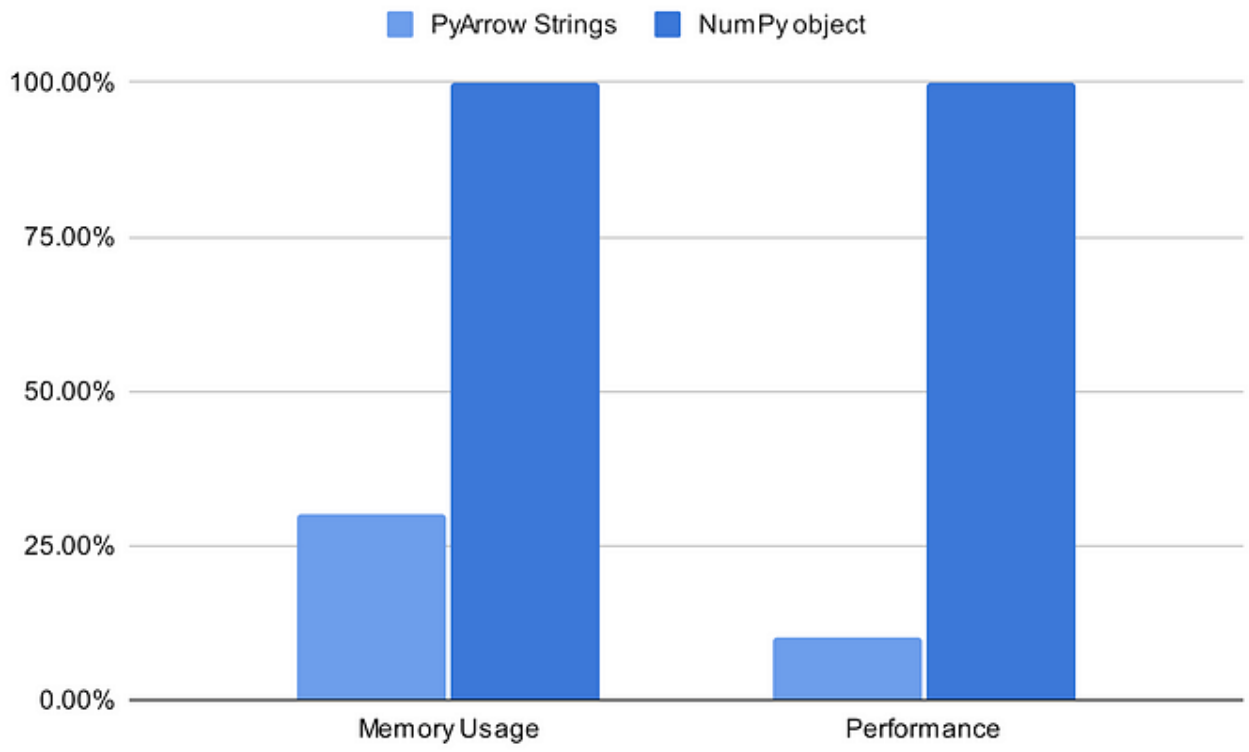
Utilizing PyArrow to Improve pandas and Dask Workflows

towardsdatascience.com/utilizing-pyarrow-to-improve-pandas-and-dask-workflows-2891d3d96d2b

Patrick Hoefler June 6, 2023



All images were created by the author

Introduction

This post investigates where we can use PyArrow to improve our pandas and Dask workflows right now. General support for PyArrow dtypes was added with pandas 2.0 to pandas and Dask. This solves a bunch of long-standing pains for users of both libraries. pandas users often complain to me that pandas does not support missing values in arbitrary dtypes or that non-standard dtypes are not very well supported. A particularly annoying problem for Dask users is running out of memory with large datasets. PyArrow backed string columns consume up to 70% less memory compared to NumPy object columns and thus have the potential to mitigate this problem as well as providing a huge performance improvement.

Support for PyArrow dtypes in pandas, and by extension Dask, is still relatively new. I would recommend caution when opting into the PyArrow `dtype_backend` until at least pandas 2.1 is released. Not every part of both APIs is optimized yet. You should be able to get a big

improvement in certain workflows though. This post will go over a couple of examples where I'd recommend switching to PyArrow right away, because it already provides huge benefits.

Dask itself can benefit in various ways from PyArrow dtypes. We will investigate how PyArrow backed strings can easily mitigate the pain point of running out of memory on Dask clusters and how we can improve performance through utilizing PyArrow.

I am part of the pandas core team and was heavily involved in implementing and improving PyArrow support in pandas. I've recently joined Coiled where I am working on Dask. One of my tasks is improving the PyArrow integration in Dask.

General overview of PyArrow support

PyArrow dtypes were initially introduced in pandas 1.5. The implementation was experimental and I wouldn't recommend using it on pandas 1.5.x. Support for them is still relatively new. pandas 2.0 provides a huge improvement, including making opting into PyArrow backed DataFrames easy. We are still working on supporting them properly everywhere, and thus they should be used with caution until at least pandas 2.1 is released. Both projects work continuously to improve support throughout Dask and pandas.

We encourage users to try them out! This will help us to get a better idea of what is still lacking support or is not fast enough. Giving feedback helps us improve support and will drastically reduce the time that is necessary to create a smooth user experience.

Dataset

We will use the taxi dataset from New York City that contains all Uber and Lyft rides. It has some interesting attributes like price, tips, driver pay and many more. The dataset can be found here (see terms of service) and is stored in parquet files. When analyzing Dask queries, we will use a publicly available S3 bucket to simplify our queries: `s3://coiled-datasets/uber-lyft-tlc/`. We will use the dataset from December 2022 for our pandas queries, since this is the maximum that fits comfortably into memory on my machine (24GB of RAM). We have to avoid stressing our RAM usage, since this might introduce side effects when analyzing performance.

We will also investigate the performance of `read_csv`. We will use the *Crimes in Chicago* dataset that can be found here.

Dask cluster

There are various different options to set up a Dask cluster, see the Dask documentation for a non-exhaustive list of deployment options. I will use Coiled to create a cluster on AWS with 30 machines through:

```
import coiled

cluster = coiled.Cluster(    n_workers=,    name=,    region=,      worker_vm_type=,)
```

Coiled is connected to my AWS account. It creates the cluster within my account and manages all resources for me. 30 machines are enough to operate on our dataset comfortably. We will investigate how we can reduce the required number of workers to 15 through some small modifications.

pandas `StringDtype` backed by PyArrow

We begin with a feature that was originally introduced over 3 years ago in pandas 1.0. Setting the dtype in pandas or Dask to `string` returns an object with `StringDtype`. This feature is relatively mature and should provide a smooth user experience.

Historically, pandas represented string data through NumPy arrays with dtype `object`. NumPy object data is stored as an array of pointers pointing to the actual data in memory. This makes iterating over an array containing strings very slow. pandas 1.0 initially introduced said `StringDtype` that allowed easier and consistent operations on strings. This dtype was still backed by Python strings and thus, wasn't very performant either. Rather, it provided a clear abstraction of string data.

pandas 1.3 finally introduced an enhancement to create an efficient string dtype. This datatype is backed by PyArrow arrays. PyArrow provides a data structure that enables performant and memory efficient string operations. Starting from that point on, users could use a string dtype that was contiguous in memory and thus very fast. This dtype can be requested through `string[pyarrow]`. Alternatively, we can request it by specifying `string` as the dtype and setting:

```
pd.options.mode.string_storage =
```

Since Dask builds on top of pandas, this string dtype is available here as well. On top of that, Dask offers a convenient option that automatically converts all string-data to `string[pyarrow]`.

```
dask.config.({: })
```

This is a convenient way of avoiding NumPy object dtype for string columns. Additionally, it has the advantage that it creates PyArrow arrays natively for I/O methods that operate with Arrow objects. On top of providing huge performance improvements, PyArrow strings consume significantly less memory. An average Dask DataFrame with PyArrow strings consumes around 33–50% of the original memory compared to NumPy object. This solves the biggest pain point for Dask users that is running out of memory when operating on large datasets. The option enables global testing in Dask's test suite. This ensures that PyArrow backed strings are mature enough to provide a smooth user experience.

Let’s look at a few operations that represent typical string operations. We will start with a couple of pandas examples before switching over to operations on our Dask cluster.

We will use `df.convert_dtypes` to convert our object columns to PyArrow string arrays. There are more efficient ways of getting PyArrow dtypes in pandas that we will explore later. We will use the Uber-Lyft dataset from December 2022, this file fits comfortably into memory on my machine.

```
import pandas as pd

pd.options.mode.string_storage = "pyarrow"

df = pd.read_parquet(    ,    columns=[        ,         ,         ,         ,
,    ],)df = df.convert_dtypes(    convert_boolean=,     convert_floating=,
convert_integer=,)
```

Our DataFrame has NumPy dtypes for all non-string columns in this example. Let’s start with filtering for all rides that were operated by Uber.

```
df[df[] == ]
```

This operation creates a mask with True/False values that specify whether Uber operated a ride. This doesn’t utilize any special string methods, but the equality comparison dispatches to PyArrow. Next, we will use the String accessor that is implemented in pandas and gives you access to all kinds of string operations on a per-element basis. We want to find all rides that were dispatched from a base starting with `"B028"`.

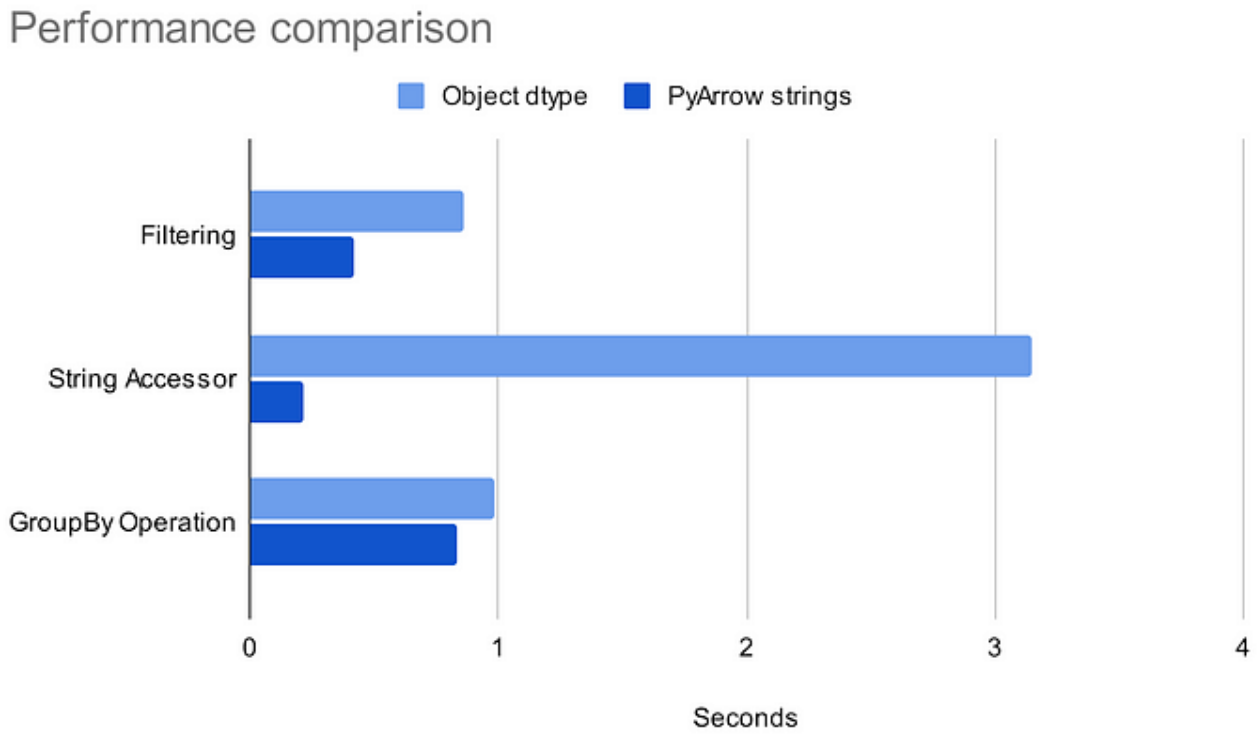
```
df[df[]..startswith()]
```

`startswith` iterates over our array and checks whether every string starts with the specified substring. The advantage of PyArrow is easy to see. The data are contiguous in memory, which means that we can efficiently iterate over them. Additionally, these arrays have a second array with pointers that point to the first memory address of every string, which makes computing the starting sequence even faster.

Finally, we look at a `GroupBy` operation that groups over PyArrow string columns. The calculation of the groups can dispatch to PyArrow as well, which is more efficient than factorizing over a NumPy object array.

```
df.groupby(    [, ]).mean(numeric_only=)
```

Let’s look at how these operations stack up against DataFrames where string columns are represented by NumPy object dtype.



The results are more or less as we expected. The string based comparisons are significantly faster when performed on PyArrow strings. Most string accessors should provide a huge performance improvement. Another interesting observation is memory usage, it is reduced by roughly 50% compared to NumPy object dtype. We will take a closer look at this with Dask.

Dask mirrors the pandas API and dispatches to pandas for most operations. Consequently, we can use the same API to access PyArrow strings. A convenient option to request these globally is the option mentioned above, which is what we will use here:

```
dask.config.({: })
```

One of the biggest benefits of this option during development is that it enables easy testing of PyArrow strings globally in Dask to make sure that everything works smoothly. We will utilize the Uber-Lyft dataset for our explorations. The dataset takes up around 240GB of memory on our cluster. Our initial cluster has 30 machines, which is enough to perform our computations comfortably.

```
import dask
import dask.dataframe as dd
from distributed import wait

dask.config.set({"dataframe.convert-string": True})

df = dd.read_parquet(    ,    storage_options={: },)df = df.persist()wait(df)
```

We persist the data in memory so that I/O performance does not influence our performance measurements. Our data is now available in memory, which makes access fast. We will perform computations that are similar to our pandas computations. One of the main goals is to show that the benefits from pandas will translate to computations in a distributed environment with Dask.

One of the first observations is that the DataFrame with PyArrow backed string columns consumes only 130GB of memory, only half of what it consumed with NumPy object columns. We have only a few string columns in our DataFrame, which means that the memory savings for string columns are actually higher than around 50% when switching to PyArrow strings. Consequently, we will reduce the size of our cluster to 15 workers when performing our operations on PyArrow string columns.

```
cluster.scale()
```

We measure the performance of the mask-operation and one of the String accessors together through subsequent filtering of the DataFrame.

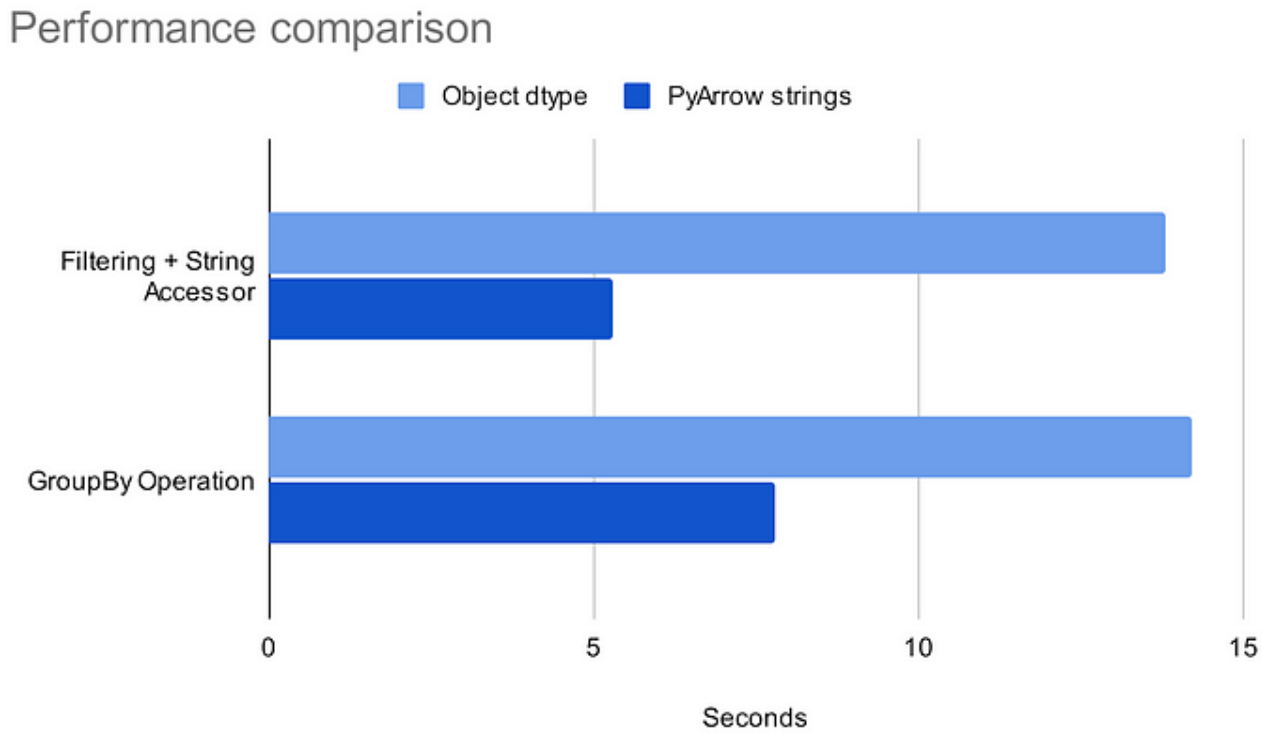
```
df = df[df[] == ]df = df[df[]..startswith()]df = df.persist()wait(df)
```

We see that we can use the same methods as in our previous example. This makes transitioning from pandas to Dask relatively easy.

Additionally, we will again compute a `GroupBy` operation on our data. This is significantly harder in a distributed environment, which makes the results more interesting. The previous operations parallelize relatively easy onto a large cluster, while this is harder with `GroupBy`.

```
df = df.groupby(
    ["hvfhs_license_num", "dispatching_base_num"]
).mean(numeric_only=True)

df = df.persist()wait(df)
```



We get nice improvements by factors of 2 and 3. This is especially intriguing since we reduced the size of our cluster from 30 machines to 15, reducing the cost by 50%. Subsequently, we also reduced our computational resources by a factor of 2, which makes our performance improvement even more impressive. Thus, the performance improved by a factor of 4 and 6 respectively. We can perform the same computations on a smaller cluster, which saves money and is more efficient in general, and still get a performance boost out of it.

Summarizing, we saw that PyArrow string-columns are a huge improvement when comparing them to NumPy object columns in DataFrames. Switching to PyArrow strings is a relatively small change that might improve the performance and efficiency of an average workflow that depends on string data immensely. These improvements are equally visible in pandas and Dask!

Engine keyword in I/O methods

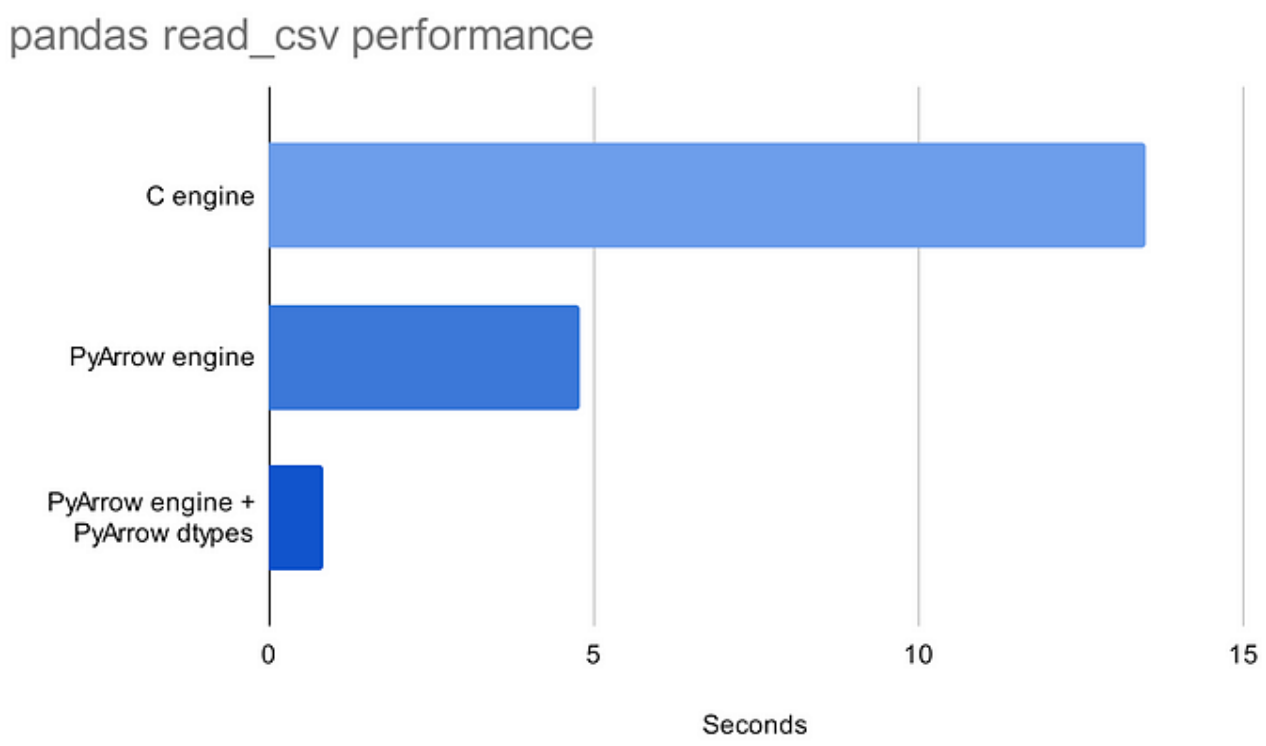
We will now take a look at I/O functions in pandas and Dask. Some functions have custom implementations, like `read_csv`, while others dispatch to another library, like `read_excel` to `openpyxl`. Some of these functions gained a new `engine` keyword that enables us to dispatch to `PyArrow`. The PyArrow parsers are multithreaded by default and hence, can provide a significant performance improvement.

```
pd.read_csv(, engine=)
```

This configuration will return the same results as the other engines. The only difference is that PyArrow is used to read the data. The same option is available for `read_json`. The PyArrow-engines were added to provide a faster way of reading data. The improved speed is only one of the advantages. The PyArrow parsers return the data as a PyArrow Table. A PyArrow Table provides built-in functionality to convert to a pandas `DataFrame`. Depending on the data, this might require a copy while casting to NumPy (string, integers with missing values, ...), which brings an unnecessary slowdown. This is where the PyArrow `dtype_backend` comes in. It is implemented as an `ArrowExtensionArray` class in pandas, which is backed by a PyArrow ChunkedArray. As a direct consequence, the conversion from a PyArrow Table to pandas is extremely cheap since it does not require any copies.

```
pd.read_csv(    ,     engine=,     dtype_backend=,)
```

This returns a `DataFrame` that is backed by PyArrow arrays. pandas isn't optimized everywhere yet, so this can give you a slowdown in follow-up operations. It might be worth it if the workload is particularly I/O heavy. Let's look at a direct comparison:



We can see that PyArrow-engine and PyArrow dtypes provide a 15x speedup compared to the C-engine.

The same advantages apply to Dask. Dask wraps the pandas csv reader and hence, gets the same features for free.

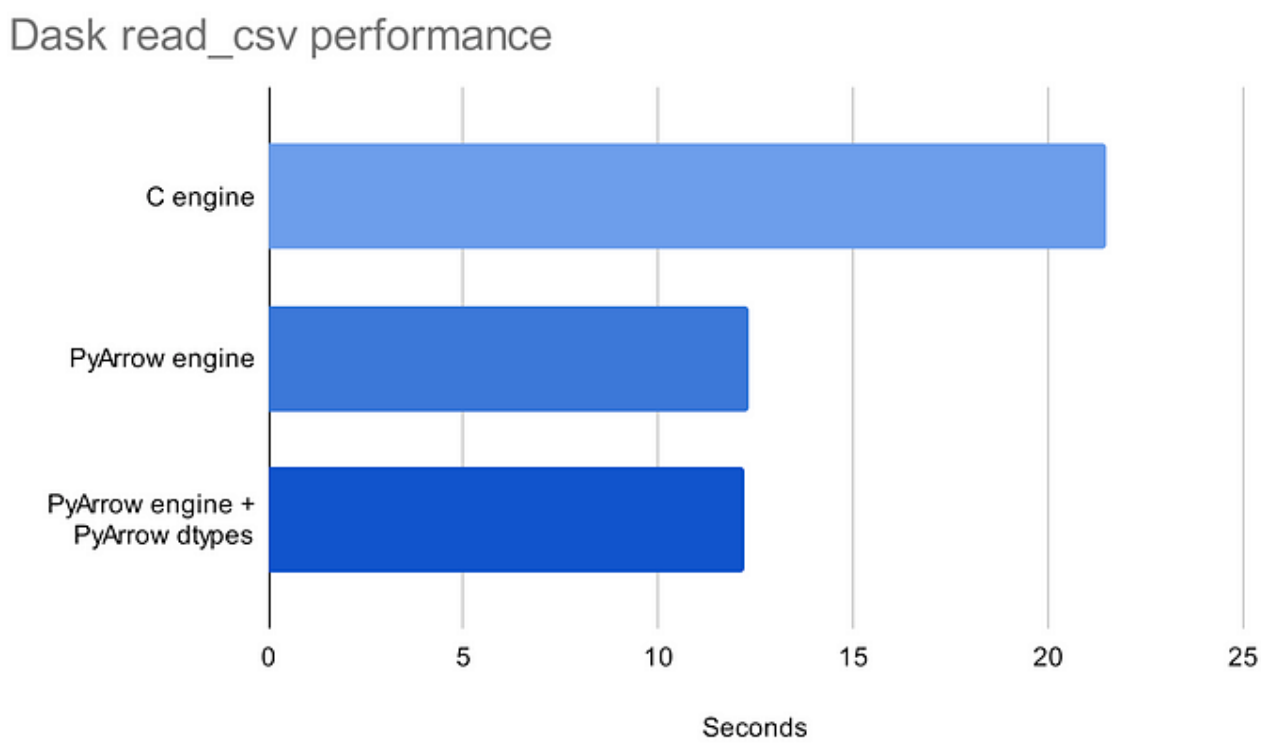
The comparison for Dask is a bit more complicated. Firstly, my example reads the data from my local machine while our Dask examples will read the data from a S3 bucket. Network speed will be a relevant component. Also, distributed computations have some overhead that we have to account for.

We are purely looking for speed here, so we will read some timeseries data from a public S3 bucket.

```
import dask.dataframe as dd
from distributed import wait

df = dd.read_csv(    ,    storage_options={: },    engine=,    parse_dates=[],)df =
df.persist()wait(df)
```

We will execute this code-snippet for `engine="c"`, `engine="pyarrow"` and additionally `engine="pyarrow"` with `dtype_backend="pyarrow"`. Let's look at some performance comparisons. Both examples were executed with 30 machines on the cluster.



The PyArrow-engine runs around 2 times as fast as the C-engine. Both implementations used the same number of machines. The memory usage was reduced by 50% with the PyArrow `dtype_backend`. The same reduction is available if only object columns are converted to PyArrow strings, which gives a better experience in follow-up operations.

We've seen that the Arrow-engines provide significant speedups over the custom C implementations. They don't support all features of the custom implementations yet, but if your use-case is compatible with the supported options, you should get a significant speedup for free.

The case with the PyArrow `dtype_backend` is a bit more complicated. Not all areas of the API are optimized yet. If you spend a lot of time processing your data outside I/O functions, then this might not give you what you need. It will speed up your processing if your workflow spends a lot of time reading the data.

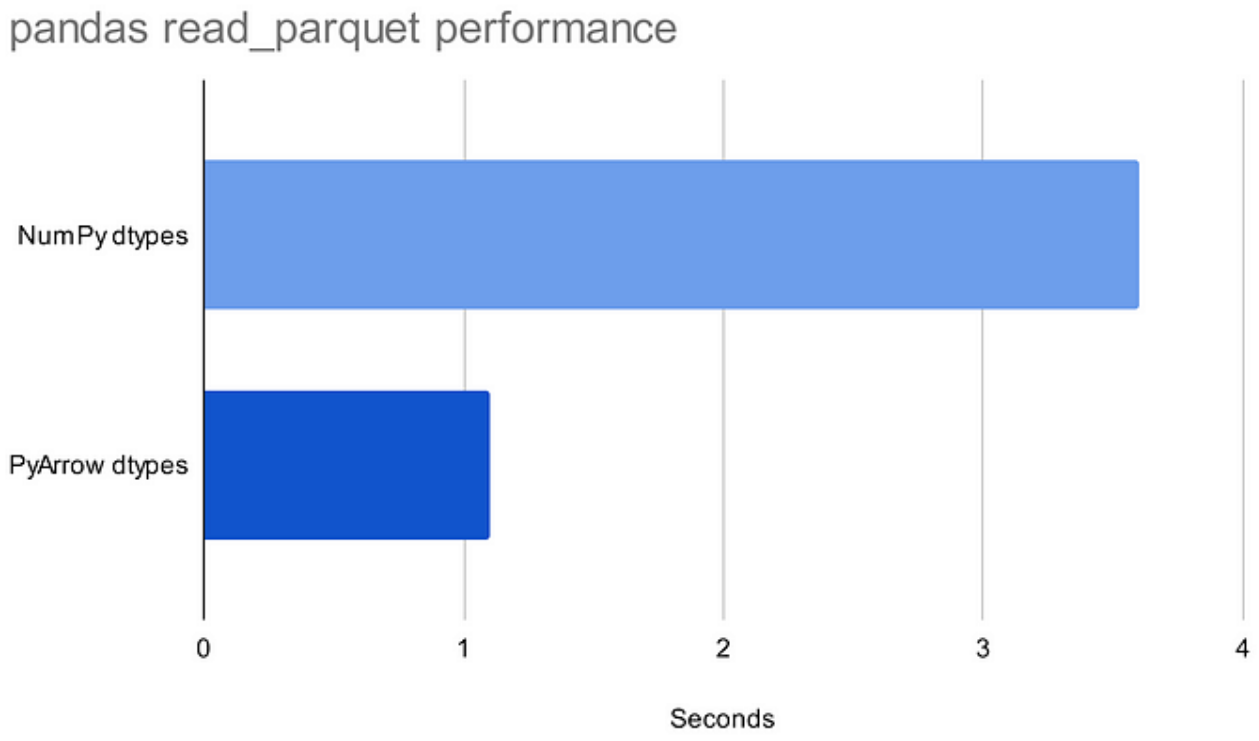
dtype_backend in PyArrow-native I/O readers

Some other I/O methods have an engine keyword as well. `read_parquet` is the most popular example. The situation is a bit different here though. These I/O methods were already using the PyArrow engine by default. So the parsing is as efficient as possible. One other potential performance benefit is the usage of the `dtype_backend` keyword. Normally, PyArrow will return a PyArrow table which is then converted to a pandas DataFrame. The PyArrow dtypes are converted to their NumPy equivalent. Setting `dtype_backend="pyarrow"` avoids this conversion. This gives a decent performance improvement and saves a lot of memory.

Let's look at one pandas performance comparison. We read the Uber-Lyft taxi data from December 2022.

```
pd.read_parquet()
```

We read the data with and without `dtype_backend="pyarrow"`.



We can easily see that the most time is taken up by the conversion after the reading of the Parquet file was finished. The function runs 3 times as fast when avoiding the conversion to NumPy dtypes.

Dask has a specialized implementation for `read_parquet` that has some advantages tailored to distributed workloads compared to the pandas implementation. The common denominator is that both functions dispatch to PyArrow to read the parquet file. Both have in common that the data are converted to NumPy dtypes after successfully reading the file. We are reading the whole Uber-Lyft dataset, which consumes around 240GB of memory on our cluster.

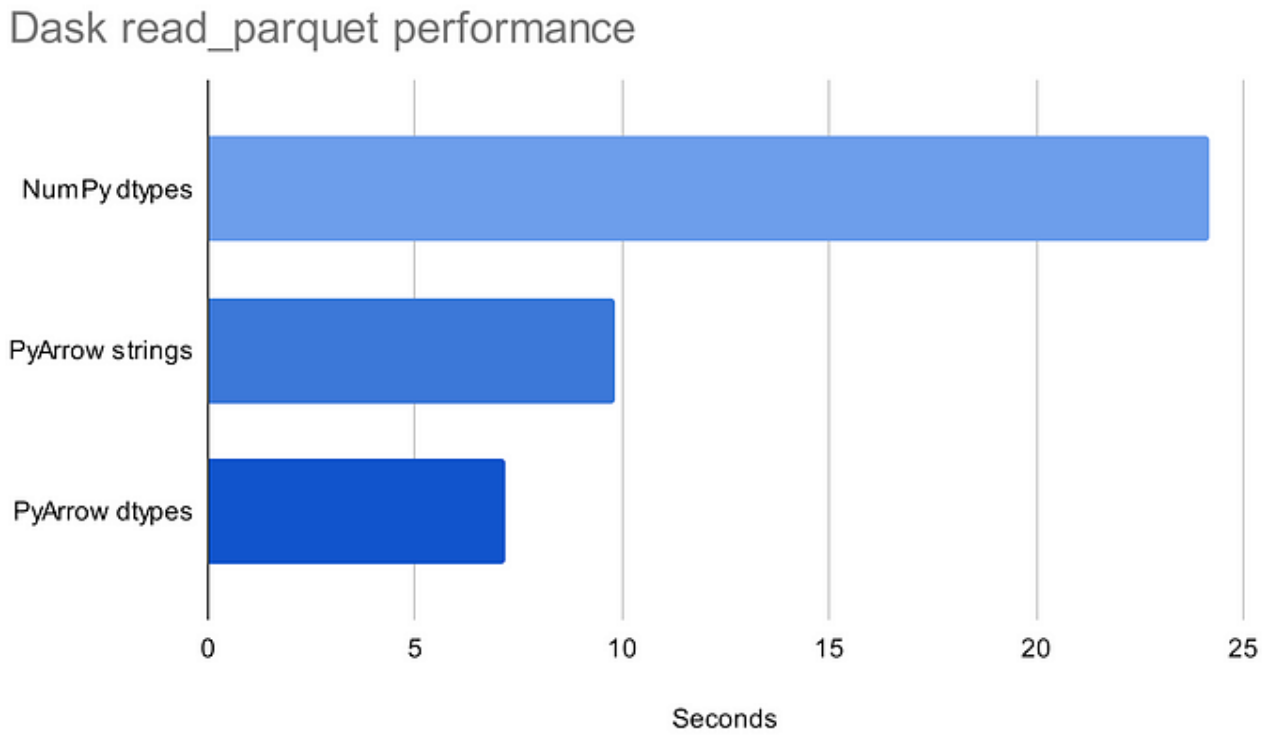
```
import dask.dataframe as dd
from distributed import wait

df = dd.read_parquet(    ,    storage_options={: },)df = df.persist()wait(df)
```

We read the dataset in 3 different configurations. First with the default NumPy dtypes, then with the PyArrow string option turned on:

```
dask.config.({: })
```

And lastly with `dtype_backend="pyarrow"`. Let's look at what this means performance-wise:



Similar to our pandas example, we can see that converting to NumPy dtypes takes up a huge chunk of our runtime. The PyArrow dtypes give us a nice performance improvement. Both PyArrow configurations use half of the memory that the NumPy dtypes are using.

PyArrow-strings are a lot more mature than the general PyArrow `dtype_backend`. Based on the performance chart we got, we get roughly the same performance improvement when using PyArrow strings and NumPy dtypes for all other dtypes. If a workflow does not work well enough on PyArrow dtypes yet, I'd recommend enabling PyArrow strings only.

Conclusion

We have seen how we can leverage PyArrow in pandas in Dask right now. PyArrow backed string columns have the potential to impact most workflows in a positive way and provide a smooth user experience with pandas 2.0. Dask has a convenient option to globally avoid NumPy object dtype when possible, which makes opting into PyArrow backed strings even easier. PyArrow also provides huge speedups in other areas where available. The PyArrow `dtype_backend` is still pretty new and has the potential to cut I/O times significantly right now. It is certainly worth exploring whether it can solve performance bottlenecks. There is a lot of work going on to improve support for general PyArrow dtypes with the potential to speed up an average workflow in the near future.

There is a current proposal in pandas to start inferring strings as PyArrow backed strings by default starting from pandas 3.0. Additionally, it includes many more areas where leaning more onto PyArrow makes a lot of sense (e.g. Decimals, structured data, …). You can read

up on the proposal here.

Thank you for reading. Feel free to reach out in the comments to share your thoughts and feedback about PyArrow support in both libraries. I will write follow up posts focused on this topic and pandas in general. Follow me on Medium if you like to read more about pandas and Dask.